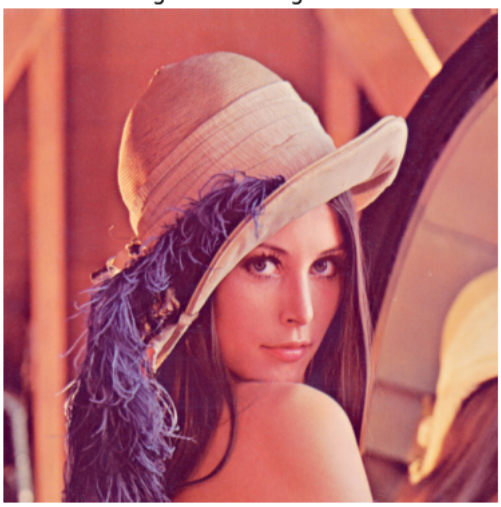
Exercise 2.1

Task 1 - Read and display the image.

```
from PIL import Image
import matplotlib.pyplot as plt

image_path = "/content/drive/MyDrive/SEM6_AI/lenna.png"

img = Image.open(image_path)

plt.imshow(img)
plt.axis("off")
plt.title("Image from Google Drive")
plt.show()
```

Image from Google Drive



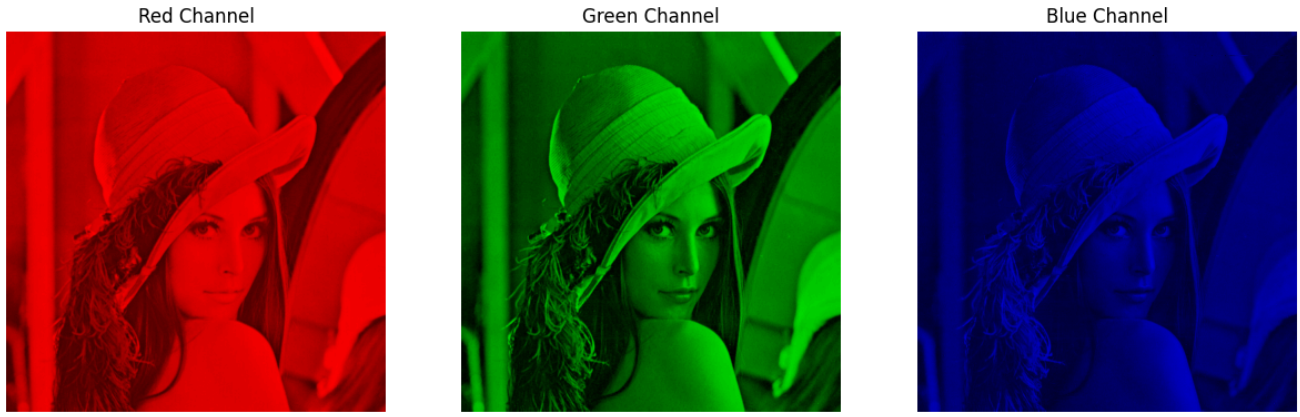
Task 2 - Display only the top left corner of 100x100 pixels.

```
import numpy as np

img_array = np.array(img)

top_left = img_array[0:100, 0:100]

plt.imshow(top_left)
plt.axis("off")
plt.title("Top-Left 100x100 Pixels")
plt.show()
```

Top-Left 100x100 Pixels

Task 3 - Show the three color channels (R, G, B).

```
import matplotlib.pyplot as plt
import numpy as np

# Assuming R, G, B channels and img_array are already defined

# Create RGB images for each channel
red_img = np.zeros_like(img_array)
red_img[:, :, 0] = R

green_img = np.zeros_like(img_array)
green_img[:, :, 1] = G

blue_img = np.zeros_like(img_array)
blue_img[:, :, 2] = B

# Display side by side
plt.figure(figsize=(15, 5))  # width x height in inches

# Red channel
plt.subplot(1, 3, 1)  # 1 row, 3 columns, first subplot
plt.imshow(red_img)
plt.title("Red Channel")
plt.axis("off")

# Green channel
plt.subplot(1, 3, 2)  # 1 row, 3 columns, second subplot
plt.imshow(green_img)
plt.title("Green Channel")
plt.axis("off")

# Blue channel
plt.subplot(1, 3, 3)  # 1 row, 3 columns, third subplot
plt.imshow(blue_img)
plt.title("Blue Channel")
plt.axis("off")

plt.show()
```



Task 4 - Modify the top 100 × 100 pixels to a value of 210 and display the resulting image.

```
modified_img = img_array.copy()

modified_img[0:100, 0:100, :] = 210

plt.imshow(modified_img)
plt.axis("off")
plt.title("Modified Image (Top-Left 100x100 = 210)")
plt.show()
```

Exercise 2

Task 1 - Load and display a grayscale image.

```
from PIL import Image
import matplotlib.pyplot as plt

image_path = "/content/drive/MyDrive/SEM6_AI/cameraman.png"

img = Image.open(image_path)

plt.imshow(img, cmap='gray')
plt.axis("off")
plt.title("Image from Google Drive")
plt.show()
```

Image from Google Drive

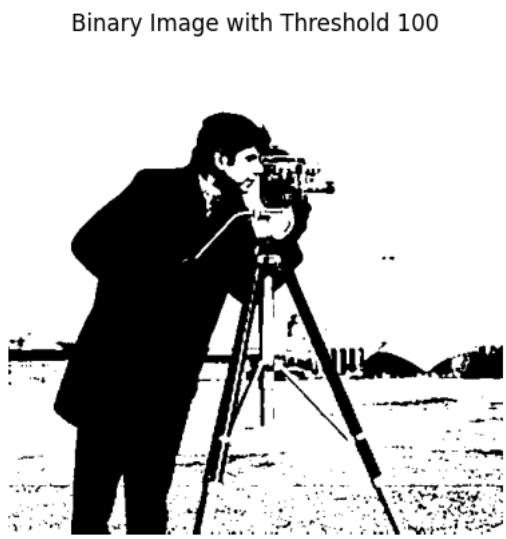
Task 2 - Extract and display the middle section of the image (150 pixels).

```
img_array = np.array(img)

# Get image dimensions
height, width = img_array.shape

# Calculate the center crop coordinates
crop_size = 150
center_y = height // 2
center_x = width // 2

start_y = center_y - crop_size // 2
end_y = center_y + crop_size // 2
start_x = center_x - crop_size // 2
end_x = center_x + crop_size // 2

# Extract the central 150x150 section
cropped_img = img_array[start_y:end_y, start_x:end_x]

# Display the cropped image
plt.imshow(cropped_img, cmap='gray')
plt.title("Center 150-pixel Section")
plt.axis("off")
plt.show()
```

Center 150-pixel Section

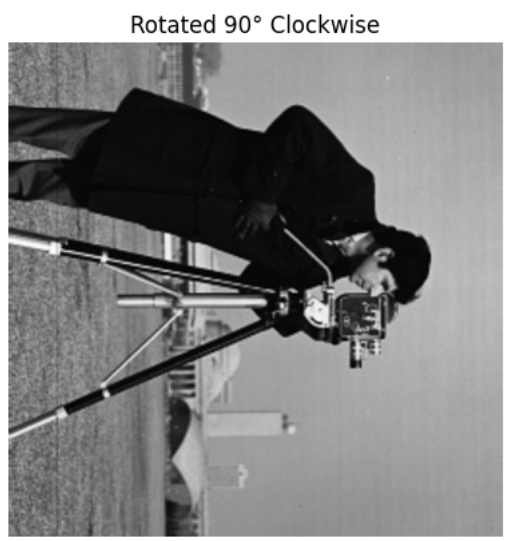
Task 3 - Apply a simple threshold to the image (e.g., set all pixel values below 100 to 0).

```
# Apply threshold
threshold_value = 100
binary_img = np.where(img_array < threshold_value, 0, 255)  # below 100 -> 0, above 100 -> 255

# Display the binary image
plt.imshow(binary_img, cmap='gray')
plt.title("Binary Image with Threshold 100")
plt.axis("off")
plt.show()
```



Task 4 - Rotate the image 90 degrees clockwise and display the result.

```
# Rotate 90 degrees clockwise
rotated_img = img.rotate(-90)  # negative for clockwise rotation

# Display
plt.imshow(rotated_img, cmap='gray')
plt.title("Rotated 90° Clockwise")
plt.axis("off")
plt.show()
```



Task 5 - Convert the grayscale image to an RGB image.

```
img_array = np.array(img)

rgb_array = np.stack((img_array,)*3, axis=-1)  # shape becomes (height, width, 3)

# Convert back to Image for display (optional)
rgb_img = Image.fromarray(rgb_array)
```

```
# Display
plt.imshow(rgb_img, cmap='gray')
plt.title("Grayscale Converted to RGB")
plt.axis("off")
plt.show()
```

Exercise 3

Task 1 - Load and Prepare Data

```
# Fetching an image
image_path = "/content/drive/MyDrive/SEM6_AI/nikisha.jpeg"

img = Image.open(image_path)
gray_img = img.convert("L")
gray_array = np.array(gray_img)
print("Grayscale image shape:", gray_array.shape)

data = gray_array.astype(float)
print("Data shape for covariance:", data.shape)
```

```
Grayscale image shape: (1203, 1024)
Data shape for covariance: (1203, 1024)
```

```
# Centering the data
mean_per_column = np.mean(data, axis=0)
data_centered = data - mean_per_column

print("Mean of centered data (should be ~0):", np.mean(data_centered))
```

```
Mean of centered data (should be ~0): -3.3784741883190134e-15
```

```
# Calculating covariance of the standardized data
cov_matrix = np.cov(data_centered, rowvar=False)  # columns are features

print("Covariance matrix shape:", cov_matrix.shape)
```

```
Covariance matrix shape: (1024, 1024)
```

Task 2 - Eigen Decomposition and Identifying Principal Components

```
# Computing Eigen Values and Eigen Vectors
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)  # use eigh for symmetric matrices like covariance

print("Eigenvalues shape:", eigenvalues.shape)
print("Eigenvectors shape:", eigenvectors.shape)
```

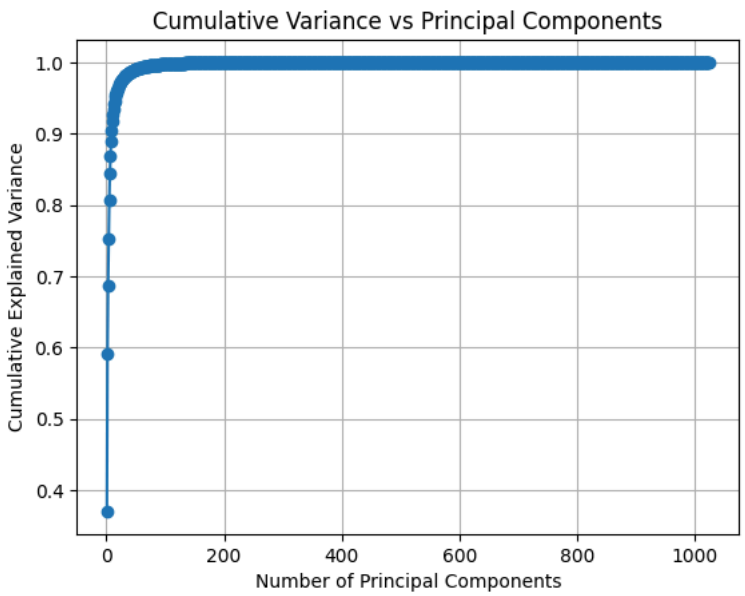
```
Eigenvalues shape: (1024,)
Eigenvectors shape: (1024, 1024)
```

```
# Sorting eigenvalues in descending order
sorted_indices = np.argsort(eigenvalues)[::-1]  # indices of largest -> smallest
eigenvalues_sorted = eigenvalues[sorted_indices]
eigenvectors_sorted = eigenvectors[:, sorted_indices]

print("Top 5 eigenvalues:", eigenvalues_sorted[:5])
```

```
Top 5 eigenvalues: [1183667.60226799  706657.86089228  306359.4607936   209432.26357967
  171625.06963162]
```

```
# Identifying Principal Component
k=5
top_k_eigenvectors = eigenvectors_sorted[:, :k]

print("Shape of top k eigenvectors:", top_k_eigenvectors.shape)

# Compute explained variance ratio
explained_variance = eigenvalues_sorted / np.sum(eigenvalues_sorted)
cumulative_variance = np.cumsum(explained_variance)

# Plot cumulative variance
plt.plot(np.arange(1, len(cumulative_variance)+1), cumulative_variance, marker='o')
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("Cumulative Variance vs Principal Components")
plt.grid(True)
plt.show()
```

```
Shape of top k eigenvectors: (1024, 5)
```



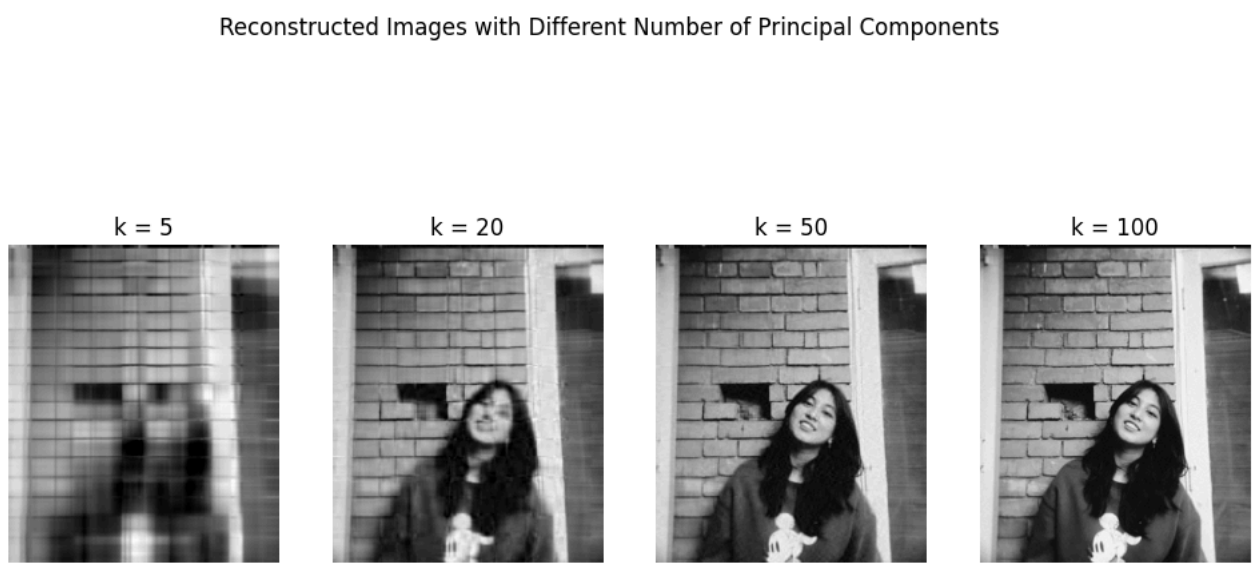
Task 3 - Reconstruction and Experiment

```
# Project the centered data onto the top k eigenvectors
low_dim_data = np.dot(data_centered, top_k_eigenvectors)  # shape: (height, k)

print("Low-dimensional representation shape:", low_dim_data.shape)

# Reconstruct by projecting back to original space
reconstructed_data = np.dot(low_dim_data, top_k_eigenvectors.T)  # shape: (height, width)

# Add back the mean to reverse centering
reconstructed_data += mean_per_column

# Convert to uint8 for display
reconstructed_img = np.clip(reconstructed_data, 0, 255).astype(np.uint8)
```

```
Low dimensional representation shape: (1203, 5)
```

```
# Experimenting with different numbers of PCs
k_values = [5, 20, 50, 100]  # number of PCs for experiments
reconstructed_images = []

for k in k_values:
    # Select top k eigenvectors
    top_k_eig = eigenvectors_sorted[:, :k]

    # Project and reconstruct
    low_dim = np.dot(data_centered, top_k_eig)
    recon = np.dot(low_dim, top_k_eig.T) + mean_per_column
    recon_img = np.clip(recon, 0, 255).astype(np.uint8)
    reconstructed_images.append(recon_img)
```

```
# displaying the reconstructed image
plt.figure(figsize=(12, 6))

for i, recon_img in enumerate(reconstructed_images):
    plt.subplot(1, len(k_values), i+1)
    plt.imshow(recon_img, cmap='gray')
    plt.title(f"k = {k_values[i]}")
    plt.axis('off')

plt.suptitle("Reconstructed Images with Different Number of Principal Components")
plt.show()

# evaluating the result
original = gray_array.astype(float)

for i, recon_img in enumerate(reconstructed_images):
    mse = np.mean((original - recon_img.astype(float))**2)
    print(f"MSE with k={k_values[i]} PCs: {mse:.2f}")
```




```
MSE with k=5 PCs: 597.97
MSE with k=20 PCs: 109.29
MSE with k=50 PCs: 30.61
MSE with k=100 PCs: 7.69
```